

Trae IDE 已支持 Doubao-Seed-1.6 模型，推理更快、准确度更高、稳定性更强

立即体验



稀土掘金

首页

AI Coding

沸点

课程

直播

活动

AI刷题

APP

插件

探索稀土掘金



创作者中心



会员

Swift内存管理和OC有什么区别

Lafar 2025-04-21 58 阅读3分钟

<<< Trae IDE 已支持 Doubao-Seed-1.6 模型，推理更快、准确度更高、稳定性更强 >>>

Swift 和 Objective-C 的内存管理都基于 自动引用计数（ARC），但具体实现和语法细节存在显著差异。以下是两者的主要区别：

1. ARC 的强制性与灵活性

- Swift：
 - 完全依赖 ARC，开发者无法手动管理内存（没有 `retain` / `release` / `autorelease` 方法）。
 - 语法更简洁，内存管理逻辑由编译器隐式处理。
- Objective-C：
 - 支持 ARC 和 **MRC**（手动引用计数），开发者可以混合使用两种模式（例如在旧代码中）。
 - 需要显式使用 `retain`、`release` 和 `autorelease`（在 MRC 模式下）。

2. 值类型与引用类型

- Swift：
 - 值类型（结构体、枚举、元组）默认通过复制传递，不涉及引用计数，避免了循环引用问题。
 - 鼓励优先使用值类型（如 `String`、`Array` 等标准库类型均为结构体），减少内存管理复杂度。
- Objective-C：
 - 几乎全是引用类型（类对象），内存管理完全依赖引用计数。
 - 值类型（如 `CGRect`、`CGPoint`）是 C 语言结构体，不参与 ARC。

3. 循环引用的处理

- Swift：
 - 使用 `weak`（弱引用）和 `unowned`（无主引用）打破循环。
 - `weak` 必须是可选类型（`var + ?`），对象释放后自动置为 `nil`。
 - `unowned` 是非可选类型，假定对象生命周期更长，需确保不会访问已释放对象。
- Objective-C：
 - 使用 `__weak` 修饰符（弱引用）和 `__unsafe_unretained`（类似 `unowned`）。



Lafar iOS Developer

69

文章

24k

阅读

15

粉丝

关注

私信

目录

收起

- ARC 的强制性与灵活性
- 值类型与引用类型
- 循环引用的处理
- 闭包（Block）的内存管理
- 自动释放池（Autorelease Pool）
- 内存泄漏检测工具
- 其他差异

总结：核心区别表

实践建议

相关推荐

.blur()调整器用法

66阅读 · 0点赞

使用渲染管线渲染图元

102阅读 · 3点赞

widget重建

105阅读 · 0点赞

编译链接装载总结

91阅读 · 2点赞

CocoaPods 私有库Spec Repo搭建与使...

171阅读 · 0点赞

精选内容

知名开源项目AList疑似被卖，玩家炸锅...

CodeSheep · 185阅读 · 2点赞

从0到1开发一个电影网站：前端小...

WildBlue · 24阅读 · 4点赞

阿里老员工发表万字离职感言引爆内网...

程序员凌览 · 61阅读 · 0点赞

Node.js 正式发布 Amaro 1.0：原生 Typ...

大知闲闲i · 63阅读 · 1点赞

JavaScript严格模式：你真的了解它...

- `__weak` 修饰的指针会自动置 `nil`，但语法上更繁琐（需手动处理空指针）。

秋天的一阵风 · 15阅读 · 0点赞

示例对比：

less

[代码解读](#) [复制代码](#)

```
1 // Swift
2 class A {
3     weak var b: B? // 弱引用必须是可选类型
4 }
5
6 class B {
7     unowned var a: A // 无主引用必须是非可选类型
8 }
```

less

[代码解读](#) [复制代码](#)

```
1 // Objective-C
2 @interface A : NSObject
3 @property (nonatomic, weak) B *b; // 弱引用
4 @end
5
6 @interface B : NSObject
7 @property (nonatomic, unsafe_unretained) A *a; // 类似 unowned
8 @end
```

找对属于你的技术圈子
回复「进群」加入官方微信群

4. 闭包（Block）的内存管理

• Swift:

- 闭包是引用类型，若捕获 `self` 可能导致循环引用，需显式使用 捕获列表：

swift

swift

[代码解读](#) [复制代码](#)

```
1 class MyClass {
2     var closure: (() -> Void)?
3     func setup() {
4         closure = { [weak self] in
5             guard let self = self else { return }
6             self.doSomething()
7         }
8     }
9 }
```

• Objective-C:

- Block 捕获对象时默认强引用，需用 `__weak` 打破循环：

objective-c

ini

[代码解读](#) [复制代码](#)

```
1 __weak typeof(self) weakSelf = self;
2 self.block = ^{
3     __strong typeof(weakSelf) strongSelf = weakSelf;
4     [strongSelf doSomething];
5 };
```

5. 自动释放池（Autorelease Pool）

- **Swift:**

- 使用 `autoreleasepool` 包裹代码块，但通常仅在需要优化大量临时对象时使用（如循环中创建对象）：

swift

ini

[代码解读](#) [复制代码](#)

```
1 autoreleasepool {
2     for _ in 0..<10000 {
3         let temp = TempObject()
4     }
5 }
```

- **Objective-C:**

- 自动释放池更常见，尤其是与旧代码（MRC）交互时。每个 RunLoop 迭代会隐式创建和释放自动释放池。

6. 内存泄漏检测工具

- **Swift:**

- **Xcode Memory Graph Debugger:** 可视化对象引用关系，直接定位循环引用。
- **Instruments 的 Leaks 工具:** 检测未释放的内存。

- **Objective-C:**

- 依赖 **Instruments 的 Leaks 和 Allocations 工具**，但缺少 Swift 的 Memory Graph 直观性。

7. 其他差异

- **可选类型（Optional）:**

- Swift 的 `weak` 变量必须是可选类型，强制开发者处理空值，避免野指针。
- Objective-C 的 `__weak` 指针自动置 `nil`，但需手动检查空值。

- **协议与泛型:**

- Swift 的协议可以约束为 `class` 类型（用于声明弱引用委托）：

swift

swift

[代码解读](#) [复制代码](#)

```
1 protocol MyDelegate: AnyObject { /* ... */ }
2 class MyClass {
3     weak var delegate: MyDelegate?
4 }
```

- Objective-C 通过 `@protocol` 定义协议，但无法直接约束为弱引用。

总结：核心区别表

特性	Swift	Objective-C
内存管理模式	强制 ARC，无手动操作	支持 ARC 和 MRC
值类型支持	广泛使用结构体、枚举（无引用计数）	主要依赖类（引用类型）
弱引用语法	<code>weak var</code> （必须为可选类型）	<code>__weak</code> （自动置 <code>nil</code> ）
无主引用	<code>unowned</code> （非可选，需确保对象存活）	<code>__unsafe_unretained</code> （不安全）
闭包/Block 循环引用	需显式使用捕获列表（ <code>[weak self]</code> ）	需显式使用 <code>__weak</code> 和 <code>__strong</code>
工具支持	Memory Graph Debugger、Leaks	Instruments 的 Leaks、Allocations

实践建议

1. **Swift** 优先使用值类型（结构体、枚举），减少引用计数问题。
2. 注意闭包中的 `self` 捕获，始终使用 `[weak self]` 或 `[unowned self]`。
3. 利用 **Memory Graph Debugger** 快速定位循环引用。
4. 从 **Objective-C** 迁移时，需适应完全自动化的 ARC 和值类型设计。

标签： 前端

评论 0



抢首评，友善交流



0 / 1000  发送

暂无评论数据

为你推荐

Swift 中的内存管理 Sheepy8年前👁718👍9💬评论 Swift程序员
Swift笔记3：指针&内存管理 理查德森4年前👁1.6k👍12💬评论 Swift
Swift-进阶 05：内存管理 & Runtime Style_月月3年前👁1.6k👍10💬1 iOSSwift
swift 内存管理 谢君1012641年前👁63👍点赞💬评论 前端
swift指针&内存管理-引用 i_erlich2年前👁447👍2💬评论 iOSSwiftApple
Swift 内存管理 swagg3r3年前👁1.2k👍5💬评论 Swift
iOS探索RxSwift之内存管理 可乐泡枸杞2年前👁1.4k👍4💬评论 前端iOS
Swift 内存管理 (2.1 高级) 宇朋Look8年前👁1.9k👍14💬1 iOSSwift
Swift-内存管理 Muen4年前👁101👍点赞💬评论 Swift
来一次有侧重点的区分Swift与Objective-C dddwncty6年前👁13k👍114💬6 iOS
Swift-内存管理与异常处理 萌呆宝2年前👁290👍点赞💬评论 前端Swift
Swift 和 Objective-C 的区别 望落还米酒4年前👁1.6k👍点赞💬评论 iOS
iOS 内存管理机制 奉孝4年前👁6.1k👍25💬2 iOS
Swift学习（四）指针和内存管理 王飞飞不会飞3年前👁339👍点赞💬评论 Swift
Swift系列面试题总结 路过看风景4年前👁12k👍31💬2 Swift

AI助手